

Individual Game Submission

CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Sai Sumedh Kaveti

Milestone 1 - Build Game Engine Foundations

Module 3 - Game Play Foundations

Individual Game Name: “Cave Ninja” GitHub Game

Repository:

<https://github.com/CodeRockerr/CSC481-581-Group4/tree/main/Games/skaveti>

(A) Introduction

This report contains my individual milestone 1 submission and reflection on how my game ‘Cave Ninja’ utilizes the common team game engine. The milestone asked not only to design a common engine that could be used by multiple people but also required for each individual to create a game that would demonstrate the necessary game systems that were implemented. As such, my teammates and I split up the common code and game-specific code. Specifically, the code that would be in the common engine would handle window creation, rendering, entities, physics, input, collisions, and scale. My game is a platformer where the player controls a cave ninja on a rock platform in a cave. The cave ninja can move left and right, jump on the platform, fall due to gravity, and return to the beginning if the shadow wisp enemy patrols and touches the player. The game also meets the scaling requirement.

(B) Shared Engine Architecture

For milestone 1, our team worked on the shared engine library. The primary modules of the engine are as follows:

1. Engine
2. Window
3. Renderer
4. Entity
5. EntityManager

6. Physics
7. Input
8. Collision

The Engine class is used to initialize SDL and run the app. The game code uses the common library and implements game-specific logic in its main.cpp file. In turn, the engine does not have any concrete objects, such as Player, Enemy, or Platform. It only has an Entity class with such properties as position, size, velocity, color, and an optional texture. Thus, the same engine interfaces are used for each teammate's game.

The principles that our engine shared code embodies are as follows:

1. The Gravity force can be enabled for particular entities while being disabled for others.
2. The keyboard reading mechanism does not use individual SDL events but polls the keyboard state.
3. The collision resolution code is separate from collision detection in the engine.
4. The Display rendering uses scaling to zoom in and zoom out rather than just modifying the window size.
5. SDL3 is used as the core library for the project.

(C) Task 1 - Engine Initialization and Main Loop

Implementation

My game constructs the shared Engine with the title Engine engine("Cave Ninja – Milestone 1"). The engine initializes SDL and creates the application window and renderer of the required size (based on the actual screen resolution) and runs the main loop as

```
engine.run([&](float deltaTime) {  
/* game-specific update logic */  
});
```

It is notable that the game does not have its own loop which would process SDL events. Instead, all the per-frame computations and logics are placed in the callback passed to the Engine::run method. In particular, each frame the shared engine performs the following operations in order:

1. handles quit events
2. reads keyboard input
3. updates the physics

4. calls the game-specific callback
5. integrates the entities' positions
6. clears the renderer
7. renders entities
8. presents the frame

The actual screen size detected at start-up is used as the reference resolution, rather than a hardcoded value. Therefore, the game can run on different resolutions depending on the actual screen size.



(fig. 1 - 'Cave Ninja' Game Screen)

(D) Task 2 - Required Game Entities

(a) Static Entity - Platform

The platform is implemented using the shared entity mechanism with velocity = 0, gravity off, and a rock-ledge sprite texture. It is placed roughly in the middle of the play area with open cave floors on each side so that the player can walk off the end and fall due to gravity. Because of the transparent padding above the visible rock surface in the platform's sprite artwork, the platform's

logical top (for collision purposes) is defined as an offset into the sprite rather than the bounding box of the sprite.



(fig. 2 Static Entity - Platform)
[Sprite generated with the help of ChatGPT Images]

(b) Controllable Entity - Ninja

The ninja is the controllable entity in the game. The ninja is represented using a four frame sprite sheet (idle plus a three frame run cycle) and the outfit was recolored black with a red trailing scarf to make it visually distinctive. Movement of the character is fully implemented in the games code (not the engine code) the engine only exposes the position/velocity fields the game updates.



(fig. 3 Controllable Entity - Ninja)
[Sprite generated with the help of ChatGPT Images]

(c) Automatically Moving Entity – Shadow Wisp

The shadow wisp moves back and forth along a segment that lies directly atop the platform. The sine function of the elapsed time is used to determine the x-position of the shadow wisp, making the movement smooth, rather than a simple back and forth linear movement. The range of the patrol is kept narrower than the width of the platform (accounting for the transparent area of the shadow wisp sprites) so that the entire creature is always seen on the platform. The flipHorizontal flag is set depending on the sign of the shadow wisps' x-displacement from one frame to the next so that the sprite is facing the direction of travel.



(fig. 4 Automatically Moving Entity – Shadow Wisp)
[Sprite generated with the help of ChatGPT Images]

(E) Task 3 - Physics and Gravity

(a) Shared Physics Design

It is implemented as a configurable acceleration in the shared Physics system. For every entity that has affectedByGravity set to true, the engine will perform:

$$velocityY += gravity * deltaTime$$

This is done by the EntityManager, and is completely independent of any particular entity's game-specific logic.

(b) Game Configuration

In Cave Ninja, only the ninja is affected by gravity.

Entity	Gravity	Reason
Ninja	Yes	Player must jump, fall, and land on the platform
Cave Platform	No	Static world geometry

ShadowWisp	No	Scripted patrol path, constrained to the platform
Background	No	Visual environment

The jump is performed with the addition of an upward impulse of velocity $Y = -700$, after which it is affected by the force of gravity back down on every frame. When walking the character off either edge of the platform no specific falling code is executed in the context of the game code, as gravity is constantly accounted for by the shared engine.



(fig. 5 Ninja Demonstrating Gravity and Jumping)

(F) Task 4 - Keyboard Input and Player Controls

(a) Input Architecture

Gameplay inputs are read from the keyboard state via polling rather than discrete SDL events. The two main queries of the Input class provided by the engine are `Input::isKeyPressed(...)`, meant for actions that are held like moving, and `Input::isKeyJustPressed(...)`, meant for actions that should

be performed only once like jumping and toggling the scaling mode; the latter is only true for the single frame when the key switches from the up to the down state, thus preventing the action from being triggered repeatedly while the key is held down. SDL events are only used at the application level, to handle the request to close the window.

(b) Game Controls

Input	Action
Left Arrow/A	Move left
Right Arrow/D	Move right
Up Arrow /W /Space	Jump
Tab/T	Percentage scaling/ Toggle Pixel

Both the WASD keys and the arrow keys are available at the same time for movement, and the jump can be done by the W, Up, or Space key, so it doesn't matter which one the player will choose since it is not defined which is the correct one. The direction the player is facing is controlled by the flipHorizontal flag instead of using different textures.

(G) Task 5 - Collision Detection and Game Responses

(a) Shared Collision System

The shared engine provides:

```
Collision::checkCollision(  
    const Entity& a,    const  
    Entity& b  
)
```

This function does axis-aligned bounding box (AABB) collision and returns a Boolean. The engine does not define any collision response, so that it is up to each individual game to define what should happen when a collision occurs. My game has two main collision responses.

(b) Ninja-Platform Collision

When the ninja's bounding box overlaps the platform while falling or at rest where the velocityY ≥ 0 and we snap their position to the platform's visible surface and make their vertical velocity

zero so they can jump again. Since the platform's sprite has transparent padding above the actual rock so we check collision against a shrunken visible copy of the platform's boundaries of the full sprite bounds. This ensures that the ninja will land right on the drawn rock surface rather than floating above it.

(c) Ninja–Wisp Collision

Both the ninja and wisp sprites have a substantial amount of transparent padding surrounding the actual character artwork. When using the raw sprite rectangles as hitboxes, collisions were appearing to occur before the characters' artwork actually seemed to touch. To fix this, I created a paddedHitbox helper that inflates an entity's bounding box by a percentage inward on each side before passing it to the shared `Collision::checkCollision` function, so that the boxes used for collisions are closer to the visible artwork than the overall size of the sprite. On collision, the ninja's position is reset to its initial X and Y coordinates and its velocity is cleared, resulting in an immediate and obvious reaction rather than requiring a separate game over screen.



(fig. 6 Ninja – Shadow Wisp Collision (Before))



(fig. 7 Ninja Respwning after Ninja– Shadow Wisp Collision (After))

(d) Falling Beyond the Screen

The same reset behavior is triggered if the ninja walks off the platform's edge and falls past the bottom of the window, so that the game continues rather than letting the character fall offscreen forever.

(H) Task 6 - Runtime Scaling

(a) Shared Scaling Design

The engine has two modes of displaying the scale of objects: `ScaleMode::Pixel` and `ScaleMode::Percentage`. In the first case, everything is displayed in the game world as many pixels as the objects have set for themselves. In the second case, the player's screen resolution is considered: The position and size of each object, in pixels, is calculated as a percentage of the starting resolution. It is important to understand that the internal values of objects are not changed. It is only the final rendering values that are scaled. Such a solution allows the game to look the

same, regardless of the scale mode and the player's window size, but the behavior of physics and collisions will depend only on the internal values and not on the scale.

(b) Game Demonstration

Pressing Tab calls the `EntityManager::toggleScaleMode()` function, which toggles between the two rendering modes. Because of this, when Percentage mode is selected, resizing the game window causes the platform, ninja, and wisp to change size in unison, whereas in Pixel mode, they stay the same size regardless of the window size.

(I) Design Limitations and Future Improvements

For Milestone 1, there are certain features that were deliberately kept relatively lightweight for the scope of the class. Collision detection is done using padded AABB boxes rather than per-frame or even pixel-perfect hitboxes, which is sufficient for the needs of the current game, but a more feature-complete variant would have collision boxes that scale per-animation-frame for fast-moving objects. The wisp's patrol path and range are hand-tuned values rather than calculated based on the actual bounds of the platform, and required iterative development to reach reasonable values but could be automated in a future milestone. Finally, the engine does not attempt to implement more complex systems such as rigid-body physics, collision manifolds, or animation state machines - features that were deliberately left out of this particular milestone to keep the shared codebase as light as possible.

(J) Conclusion

Cave Ninja utilizes all of the described Milestone 1 requirements through the implemented shared team engine. The engine offers several shared mechanisms, including the game initialization, rendering, entities, physics, keyboard control, collision, and scaling. My particular game takes advantage of the shared code to create a controllable ninja entity, a stationary platform, a shadow-wisp enemy that patrols, jumping and falling physics with an adjustable gravity coefficient, two different collision response mechanisms, and sprites with flipping capabilities, which are scaled in either Pixel or Percentage mode. In doing so, the code satisfies each of the requirements listed in the course description. At that point, it is possible to note that the code separates the shared engine and the individual game code, which is manifested in the different ways the objects are handled: update, draw, and check collision, and the differences between what they are in the game and how they are represented in the engine. This separation ensures that the same engine code may be used for entirely different games within the team while satisfying the requirements of Milestone 1.
